PEAfowl: Perception-Enhanced Multi-View Vision-Language-Action for Bimanual Manipulation

Qingyu Fan[1], Zhaoxiang Li[2,*], Jinrui Hu[1,*], Yi Lu[2], Wang Chen[2], Qiu Shen[2], Xiao-xiao Long[2,†], Yinghao Cai[1,†], Tao Lu[1], Shuo Wang[1], and Xun Cao[2]

arXiv:2601.17885v2 [cs.CV] 24 Aug 2026

***Abstract*—Bimanual manipulation in cluttered scenes requires policies that remain stable under occlusions, viewpoint changes and scene variations. Existing vision-language-action models often lack such robustness because (i) multi-view features are fused via view-agnostic token concatenation, yielding limited cross-view spatial representations, and (ii) language is injected as global conditioning, resulting in coarse instruction grounding. In this paper, we introduce PEAfowl, a perception-enhanced multi-view VLA policy for bimanual manipulation. For spatial perception, PEAfowl predicts per-token depth distributions, performs differentiable 3D lifting, and aggregates local cross-view neighbors to form geometrically grounded, cross-view aligned representations. For language utilization, we propose to replace global conditioning with a Perceiver-style text-aware readout over frozen CLIP visual features, enabling iterative evidence accumulation. To better exploit commodity RGB-D sensing despite noisy and incomplete depth, PEAfowl's depth-distribution lifting naturally supports training-only depth distillation, where a pretrained depth teacher supervises the depth-distribution head to inject refined geometric priors without adding inference overhead. On RoboTwin 2.0 under domain-randomized setting, PEAfowl improves the strongest baseline by 23.0 pp in success rate, and physical experiments further demonstrate improved performance on the evaluated real-robot tasks. Project website: https://peafowlvla.github.io/.**

***Index Terms*— Bimanual Manipulation, Deep Learning in Grasping and Manipulation, RGB-D Perception, Vision-Language-Action Models**

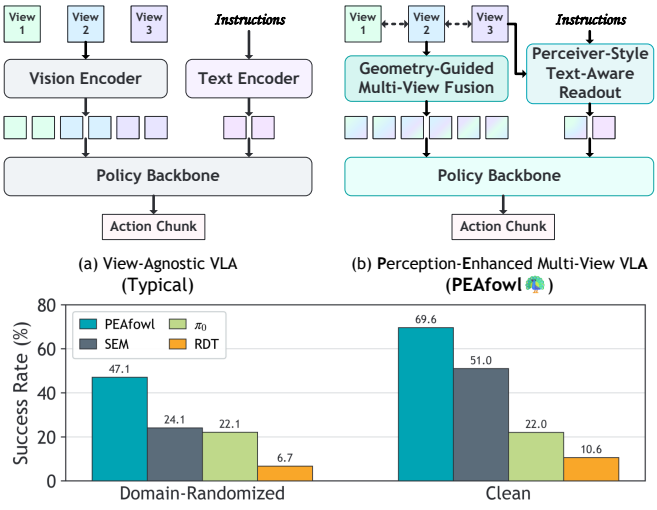


Fig. 1. **Motivation and overview of PEAfowl.** (a) Prior bimanual VLAs typically concatenate per-view visual tokens and apply global text conditioning, without explicit cross-view geometric alignment or instruction-relevant visual evidence retrieval and aggregation. (b) We propose PEAfowl, which incorporates geometry-guided multi-view fusion and a Perceiver-style text-as-query readout over frozen CLIP features. Bottom: Average success rates on RoboTwin 2.0 (nine training tasks) under Clean and Domain-Randomized settings, compared with multi-task bimanual baselines.

I. INTRODUCTION

MULTI-TASK bimanual manipulation requires a unified robot policy that integrates vision, language, and proprioception to follow diverse instructions under changing scenes and viewpoints. Recent vision-language-action (VLA) models have shown encouraging performance in single-arm manipulation [1], [2]. By leveraging large-scale robot data and web-scale vision-language supervision, a single policy can now execute a wide range of manipulation skills from natural-language prompts showing promising performance in both simulation and real-world environments.

However, multi-task bimanual manipulation remains underexplored, especially under challenging scene variations. Pretrained VLA policies often exhibit substantial performance degradation in cluttered scenes with distractors and under appearance and viewpoint variations. Generalization to unseen objects or scenes becomes particularly unstable for tasks that require coordinated bimanual motion [3]. The high-dimensional and tightly coupled bimanual action space increases the data requirements for learning robust multi-task policies. Moreover, bimanual manipulation often involves frequent self-occlusions and inter-object occlusions as well as fine-grained language instructions, which place higher demands on spatial perception and instruction-conditioned visual readout. Under limited demonstrations, improving percep-

Manuscript received: April 13, 2026; Revised: July 10, 2026; Accepted: August 11, 2026.

This paper was recommended for publication by Editor Julia Borras Sol upon evaluation of the Associate Editor and Reviewers' comments. This work was supported in part by New Generation Artificial Intelligence National Science and Technology Major Project 2025ZD0122904 and National Science and Technology Major Project 2026ZD1609003, and in part by National Natural Science Foundation of China (Grant Number:62473366, 62273342 and U23B2038).

[1]Qingyu Fan, Jinrui Hu, Yinghao Cai, Tao Lu, and Shuo Wang are with the Institute of Automation, Chinese Academy of Sciences, Beijing 100190, China (e-mail: qingyu.fan.ai@outlook.com; yinghao.cai@ia.ac.cn).

[2]Zhaoxiang Li, Yi Lu, Wang Chen, Qiu Shen, Xiao-xiao Long, and Xun Cao are with Nanjing University, Nanjing 210023, China (e-mail: xxlong@nju.edu.cn).

*Zhaoxiang Li and Jinrui Hu contributed equally to this work.

†Corresponding authors: Xiao-xiao Long and Yinghao Cai.

Author-accepted manuscript. Early Access citation: Q. Fan *et al.*, "PEAfowl: Perception-Enhanced Multi-View Vision-Language-Action for Bimanual Manipulation," *IEEE Robotics and Automation Letters*, early access, 2026, doi: 10.1109/LRA.2026.3726379. Available: https://ieeexplore.ieee.org/document/11661761.

tion efficiency and instruction-relevant evidence aggregation is therefore critical for multi-task learning and cross-scene robustness.

Bimanual manipulation platforms are inherently multi-view perception systems: multiple cameras are commonly deployed to increase workspace coverage and mitigate occlusions. These complementary views capture rich 3D geometric information that, when properly leveraged, can stabilize spatial understanding across scenes and camera configurations. However, most existing VLA pipelines treat multi-view inputs in a view-agnostic manner: each image is encoded independently, and tokens from different cameras are concatenated or stacked before being passed to a shared policy head [2], [4], [5]. Such designs do not explicitly model cross-view geometric correspondences or encourage 3D-aligned feature interaction, making the learned representations more sensitive to changes in camera poses, calibration errors, and occlusions.

Another key limitation lies in how language interacts with visual features. In many VLA architectures [2], [6], language is injected as a global conditioning vector or via a small number of text tokens appended to the visual stream, while the dominant attention computation remains vision-centric. This strategy is often sufficient for low-clutter tasks, but in multi-task, multi-object scenes it may produce unfocused, instruction-agnostic attention and weak exploitation of pre-trained vision-language alignment to identify relevant objects and spatial relations in the scene. Recent work suggests that text-aware visual feature extraction, in which language tokens explicitly query and aggregate relevant visual evidence, can improve grounding performance [7], yet such strategies have not been explored for multi-view bimanual manipulation.

In this paper, we introduce PEAfowl, a multi-view vision-language-action model with geometry- and language-guided perception for bimanual manipulation. On the spatial side, we propose a geometry-driven multi-view fusion module that (i) performs per-patch RGB-D token fusion, (ii) predicts a depth distribution for each visual token, and (iii) backprojects tokens into a shared 3D base frame to perform local 3D neighborhood aggregation across cameras. This design explicitly models cross-view geometric correspondences and endows 2D visual features with depth-aware and cross-view-aligned structure.

On the language side, PEAfowl builds upon OTTER-style text-aware visual extraction [7], and replaces global text conditioning with a Perceiver-style text-aware transformer [8]. Text tokens act as latent queries that iteratively cross-attend to per-view patch features, producing a compact set of language-conditioned visual tokens. This mechanism sharpens attention over task-relevant objects and spatial relations suitable for policy learning. Fig. 1 provides an overview of this architecture and contrasts it with a standard VLA pipeline.

Commodity RGB-D sensors provide depth measurements for real-robot manipulation, but these measurements are often noisy and incomplete. Since PEAfowl represents geometry through per-token depth distributions, completed depth can be used as a training-only supervisory signal for 3D lifting. We leverage a pretrained Camera Depth Model (CDM) [9] as a training-time depth teacher while still feeding only raw depth images to the policy at both training and test time, which introduces no test-time overhead and transfers refined geometric priors into the multi-view policy.

Both simulation and physical experiments demonstrate that PEAfowl, despite having only **312M** trainable parameters, outperforms existing bimanual VLA models and visuomotor baselines, and shows stronger robustness to novel scene appearances, workspace geometries, and language instructions. In simulation, we evaluate on 9 tasks from RoboTwin 2.0 [3] under both clean and heavily domain-randomized settings. On physical platforms, PEAfowl demonstrates improved performance, particularly on tasks requiring accurate bimanual coordination. To summarize, our contributions are:

1) We introduce a geometry-guided multi-view perception module that builds spatially-aware representations from multi-view RGB-D observations, along with the depth distillation scheme for real-world deployment.
2) We propose to replace common global text conditioning in visual-language interaction with a Perceiver-style text-as-query readout over frozen CLIP features, which produces compact instruction-conditioned tokens.
3) Extensive experiments in both simulation and real-world bimanual manipulation tasks demonstrate the benefits of geometry- and language-guided multi-view perception.

II. RELATED WORK

A. Multi-view Perception

Multi-view perception has been studied extensively in classical structure-from-motion and multi-view stereo, and more recently in neural reconstruction and multi-view transformers [10]. Most of these approaches are typically optimized for offline 3D perception and rely on heavy backbones or dense geometric supervision, which limits their suitability as lightweight perception modules tightly coupled with robot policy learning.

In robotic manipulation, multi-view observations are used to improve robustness to viewpoint variations and mitigate occlusions [11]–[14]. While these methods demonstrate the benefit of multi-view sensing, many collapse views into view-invariant embeddings or fuse per-view features through simple concatenation, where cross-view 3D consistency is not explicitly considered and depth uncertainty is largely ignored. In contrast, PEAfowl introduces a geometry-guided multi-view perception module that maintains cross-view spatial tokens under noisy depth measurements. This policy-friendly multi-view perception design explicitly leverages geometry under noisy RGB-D observations, improving robustness to viewpoint changes and scene variations.

B. Vision-Language-Action Models

VLA policies map visual observations and language instructions to actions [1]. Existing methods and datasets largely focus on single-arm manipulation. In multi-view settings, images are commonly treated as view-agnostic 2D tokens, with language injected as global conditioning. Recent bimanual VLAs still rely primarily on 2D RGB features and multi-image concatenation [5], [6]. Spatially enhanced VLAs instead inject

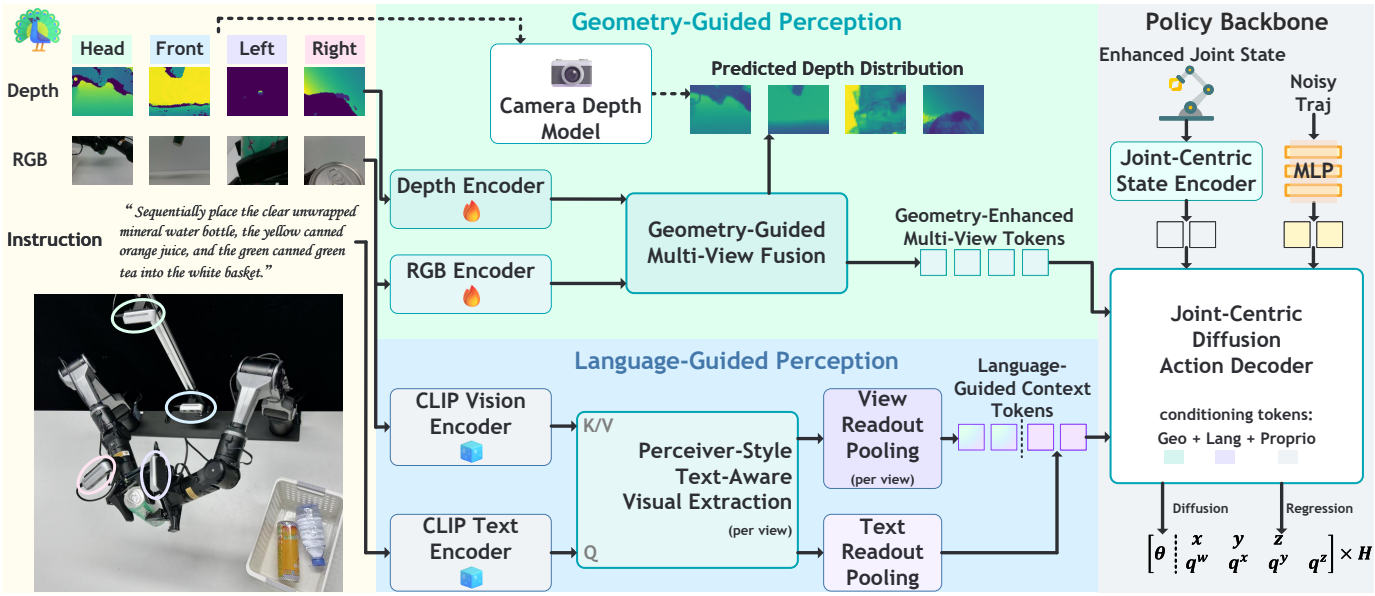


Fig. 2. **PEAfowl architecture.** PEAfowl couples geometry-guided multi-view fusion with language-guided readout to condition a SEM-style diffusion action decoder. Top: RGB–D tokens are used to predict per-token depth distributions for differentiable 3D lifting and cross-view fusion; a pretrained camera depth model supervises the depth-distribution head during training only. Bottom: Frozen CLIP features are queried by a Perceiver-style text-as-query readout and pooled into compact context tokens.

explicit 3D information into policy representations [15]. SEM aligns multi-view features and robot states in a shared base frame to form spatial representations [4]. For language conditioning, OTTER improves grounding via computing similarity scores between each instruction token and each visual feature to highlight the visual information relevant to the instruction [7]. PEAfowl combines 3D-aligned multi-view tokens with iterative text-as-query readout to enable spatial reasoning and more accurate grounding in cluttered bimanual scenes. This helps the policy more reliably disambiguate instruction-relevant targets in cluttered scenes and under scene variations.

III. METHOD

Fig. 2 presents an overview of PEAfowl. The framework integrates geometry-guided multi-view fusion with language-guided readout to produce policy-ready tokens for a joint-centric diffusion action decoder.

A. Problem Formulation

Let $\mathcal{V} = \{1, \dots, V\}$ denote the camera set. The robot receives multi-view RGB-D observations $\mathbf{o}_t \triangleq \{\mathbf{I}_t^{(v)}, \mathbf{D}_t^{(v)}\}_{v\in\mathcal{V}}$, along with proprioceptive state $\mathbf{s}_t \in \mathbb{R}^{d_s}$ and instruction ℓ. Our goal is to learn a bimanual policy π_ϕ that predicts an action chunk of horizon H conditioned on $(\mathbf{o}_t, \ell, \mathbf{s}_t)$:

$$\hat{\mathbf{a}}_{t:t+H-1} = \pi_\phi(\mathbf{o}_t, \ell, \mathbf{s}_t) \tag{1}$$

where $\mathbf{a}_t = [\mathbf{a}_t^L; \mathbf{a}_t^R] \in \mathbb{R}^{d_a}$ concatenates the left- and right-arm control commands.

B. Geometry-Guided Multi-View Fusion

Bimanual manipulation in cluttered scenes can be easily affected by self-occlusions and inter-object occlusions, which motivates explicit 3D alignment for multi-view fusion. Given multi-view RGB-D pyramids, our geometry-guided fusion produces cross-view-aligned tokens via (i) per-patch RGB-D fusion, (ii) per-token depth distributions for differentiable 3D lifting, and (iii) cross-view neighbor aggregation in a shared base frame as shown in Fig. 3.

a) Multi-view RGB-D Feature Extraction: For each view v, we extract multi-scale RGB and depth feature pyramids with modality-specific encoders shared across cameras. The RGB encoder is initialized from Grounding-DINO [16] and depth features are extracted using a lightweight ResNet [17] to obtain $\{\mathbf{F}_{\text{rgb},l}^{(v)}\}_{l=1}^{L}$ and $\{\mathbf{F}_{\text{dep},l}^{(v)}\}_{l=1}^{L}$.

b) Tokenization across Scales: We flatten the L-level feature pyramids into RGB and depth token sequences, denoted by $\mathbf{T}_{\text{rgb}}^{(v)} = \{\mathbf{t}_{\text{rgb},n}^{(v)}\}$ and $\mathbf{T}_{\text{dep}}^{(v)} = \{\mathbf{t}_{\text{dep},n}^{(v)}\}$. Known camera intrinsics and extrinsics are used for backprojection in the subsequent 3D lifting.

c) Depth-Aware 3D Lifting: To model depth ambiguity at the patch level, we predict a discrete depth distribution $\mathbf{p}_n^{(v)}$ over B bins from co-located RGB–D token pairs, and use it to softly lift each 2D token into the robot base frame. For view v and token n, the token center $\mathbf{u}_n^{(v)}$ is backprojected at each depth bin d_i to obtain 3D samples $\mathbf{x}_{n,i}^{(v)}$. We then compute the expected 3D anchor and a depth-aware point embedding by distribution-weighted aggregation:

$$\bar{\mathbf{x}}_n^{(v)} = \sum_{i=1}^{B} p_{n,i}^{(v)}\, \mathbf{x}_{n,i}^{(v)}, \qquad \mathbf{g}_n^{(v)} = \sum_{i=1}^{B} p_{n,i}^{(v)}\, \phi(\mathbf{x}_{n,i}^{(v)}) \tag{2}$$

where $\phi(\cdot)$ projects 3D coordinates to point features. We use $\bar{\mathbf{x}}_n^{(v)}$ for cross-view matching and $\mathbf{g}_n^{(v)}$ as the geometric context for subsequent fusion.

d) Pairwise RGB-D Token Fusion: To incorporate noisy commodity depth measurements while preserving pretrained

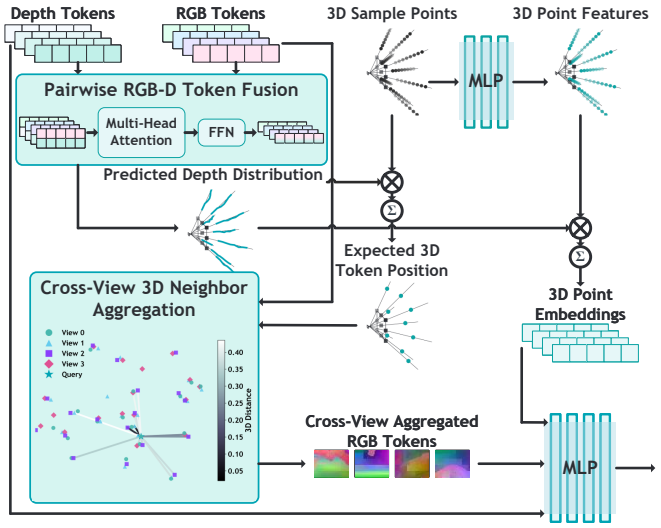


Fig. 3. **Geometry-Guided Multi-View Fusion (GGMVF).** Multi-scale RGB and depth features are tokenized, and co-located RGB-D pairs are used to predict discrete depth distributions for differentiable 3D lifting. The resulting 3D anchors enable top-K cross-view neighbor aggregation in the base frame using distance-based softmax weights and a gated residual update. Aggregated RGB tokens are fused with depth tokens and 3D point embeddings via an MLP to produce geometry-enhanced tokens.

RGB semantics, we apply a lightweight *local* fusion only on co-located RGB-D token pairs and only in the depth-distribution branch. The RGB token is projected to the depth feature dimension using a linear transformation matrix $\mathbf{W}_r$ and fused with the length-2 pair using a small attention block:

$$[\hat{\mathbf{r}}_n^{(v)}, \hat{\mathbf{d}}_n^{(v)}] = \mathrm{PairAttn}\big([\mathbf{W}_r \mathbf{t}_{\mathrm{rgb},n}^{(v)},\ \mathbf{t}_{\mathrm{dep},n}^{(v)}]\big) \tag{3}$$

By restricting attention to co-located RGB-D pairs, the fusion learns selective mixing while leaving the main RGB stream unchanged.

e) Cross-View 3D Neighbor Aggregation: Given expected 3D token anchors $\{\bar{\mathbf{x}}_n^{(v)}\}$ from Eq. (2), we aggregate information across views based on geometric proximity in the shared base frame. For each query token (v, n), we compute pairwise distances $\delta_{n,m}^{(v,w)} = \|\bar{\mathbf{x}}_n^{(v)} - \bar{\mathbf{x}}_m^{(w)}\|_2$ to tokens (w, m) from other views, select the K nearest neighbors $\mathcal{N}_n^{(v)}$, and aggregate their RGB tokens with a distance-softmax:

$$\alpha_{n,m}^{(v,w)} = \frac{\exp(-\delta_{n,m}^{(v,w)}/\tau)}{\sum_{(w',m')\in\mathcal{N}_n^{(v)}} \exp(-\delta_{n,m'}^{(v,w')}/\tau)} \tag{4}$$

where τ controls the softness of geometric matching. The aggregated feature is then computed as:

$$\mathbf{h}_n^{(v)} = \sum_{(w,m)\in\mathcal{N}_n^{(v)}} \alpha_{n,m}^{(v,w)}\, \mathbf{t}_{\mathrm{rgb},m}^{(w)} \tag{5}$$

The aggregated feature is then added to the original token via a gated residual update:

$$\tilde{\mathbf{t}}_{\mathrm{rgb},n}^{(v)} = \mathbf{t}_{\mathrm{rgb},n}^{(v)} + \gamma\, \mathbf{h}_n^{(v)} \tag{6}$$

where γ is a learnable scalar gate. This geometry-based aggregation aligns tokens that correspond to the same physical region across cameras, thereby improving robustness to occlusions and viewpoint changes. The resulting tokens are fused with depth tokens and 3D point embeddings via an MLP to form inputs to the action decoder.

C. Language-Guided Multi-View Readout

To improve instruction alignment in cluttered multi-object scenes, we replace global text conditioning with a text-aware multi-view readout over frozen CLIP features. For each view, we use a Perceiver-style extractor that iteratively updates text latents by cross-attending to CLIP patch tokens, yielding compact instruction-conditioned tokens for the policy.

a) Frozen CLIP Tokens: Given instruction ℓ and multi-view RGB images $\{\mathbf{I}^{(v)}\}_{v\in\mathcal{V}}$, we extract frozen CLIP text tokens and per-view patch tokens [18]. For denser vision-language alignment, we use the final attention-layer outputs of the CLIP vision encoder [19]:

$$\mathbf{T}_{\mathrm{txt}} = \mathrm{CLIP}_{\mathrm{txt}}(\ell) \in \mathbb{R}^{K_{\mathrm{txt}}\times D_c} \tag{7}$$

$$\mathbf{X}^{(v)} = \mathrm{CLIP}_{\mathrm{img}}^{\text{attn-last}}(\mathbf{I}^{(v)}) \in \mathbb{R}^{N_p\times D_c} \tag{8}$$

b) Perceiver-Style Text-as-Query Readout: For each view v, latent queries are initialized with text tokens and refined through M latent blocks that alternate cross-attention to patches and latent self-attention:

$$\mathbf{Z}^{(v,0)} = \mathbf{T}_{\mathrm{txt}} \tag{9}$$

$$\mathbf{Z}^{(v,m+1)} = \mathrm{LatentBlock}\Big(\mathbf{Z}^{(v,m)},\ \mathbf{X}^{(v)}\Big), m = 0, .., M-1 \tag{10}$$

which produce vision-conditioned text latents $\mathbf{Z}^{(v)} \triangleq \mathbf{Z}^{(v,M)} \in \mathbb{R}^{K_{\mathrm{txt}}\times D_c}$. ReZero-gated residuals are used for stable stacking [20].

c) Readout Tokens: We summarize tokens into a fixed number of context tokens via attention pooling:

$$\mathbf{R}^{(v)} = \mathrm{Pool}\Big(\mathbf{Z}^{(v)}\Big) \in \mathbb{R}^{R\times d},\ \mathbf{R}_{\mathrm{txt}} = \mathrm{Pool}(\mathbf{T}_{\mathrm{txt}}) \in \mathbb{R}^{R\times d} \tag{11}$$

and concatenate them as the language-guided context sequence:

$$\mathbf{S} = [\mathbf{R}_{\mathrm{txt}}; \mathbf{R}^{(1)}; \ldots; \mathbf{R}^{(V)}] \in \mathbb{R}^{(R+V\times R)\times d} \tag{12}$$

which conditions the action decoder together with geometry-enhanced tokens (Sec. III-B) and proprioception.

D. Policy Backbone

PEAfowl uses the SEM-style joint-centric policy backbone [4]. At each time step, the robot state is represented by joint-centric tokens, where each token contains the joint angle and its forward-kinematics pose in the robot base frame. A joint-centric diffusion transformer predicts an H-step bimanual trajectory conditioned on the encoded robot state, geometry-enhanced multi-view tokens, and language-guided context tokens. Compared with SEM, PEAfowl keeps the joint-centric diffusion policy unchanged and modifies the perception modules before action decoding, including the geometry-guided multi-view fusion and language-guided readout described in Secs. III-B and III-C.

E. Training Objectives

We optimize PEAfowl with a joint-centric diffusion imitation objective for policy learning and an auxiliary depth distillation loss for geometry-guided perception.

a) Diffusion Imitation Loss: The diffusion decoder predicts an H-step bimanual joint trajectory. Given a noisy input, the decoder outputs the denoised sequence $\hat{\mathbf{J}}_{t:t+H-1}$. Training minimizes a weighted diffusion loss:

$$\mathcal{L}_{\text{diff}} = \left\| (\hat{\mathbf{J}} - \mathbf{J})\, \mathbf{W} \right\|_2^2 \tag{13}$$

where $\mathbf{W}$ is a diagonal weight matrix. Following SEM [4], we add a forward-kinematics consistency term $\mathcal{L}_{\text{fk}}$ by recomputing joint poses from the predicted joint angles.

b) Depth Distillation with Camera Depth Models: Our geometry-guided branch predicts a per-token depth distribution $\mathbf{p}_n^{(v)}$ (Sec. III-B), which is well suited for depth distillation from a pretrained depth teacher to inject geometry-aware priors into the policy. We adopt the Camera Depth Model (CDM) from [9] and preprocess the training set offline to obtain refined depths $\tilde{\mathbf{D}}^{(v)} = f_{\text{CDM}}(\mathbf{I}^{(v)}, \mathbf{D}^{(v)})$, from which we derive soft depth-bin targets $\mathbf{q}_n^{(v)}$. Specifically, for each refined pixel depth d, we construct a soft target over the discretized depth bins by linearly interpolating to its two nearest bin centers, yielding a sparse two-hot distribution. These per-pixel targets are then average-pooled to token-level targets aligned with the multi-scale feature strides, producing $\mathbf{q}_n^{(v)}$ for supervising the per-token depth-distribution head. We further compute the validity weight $\omega_n^{(v)}$ as the fraction of valid pixels within each pooled region, so that unreliable or missing-depth regions contribute less to the supervision. During both training and inference, the policy input remains the raw depth $\mathbf{D}^{(v)}$; the depth teacher is used only to produce completed-depth supervision labels during training and introduces no inference overhead. The depth distribution is supervised with a validity-weighted soft-label BCE loss:

$$\mathcal{L}_{\text{depth}} = \frac{1}{VN} \sum_{v \in \mathcal{V}} \sum_{n=1}^{N} \omega_n^{(v)} \cdot \text{BCE}\Big(\mathbf{p}_n^{(v)}, \mathbf{q}_n^{(v)}\Big) \tag{14}$$

c) Total Loss: The overall training objective combines diffusion imitation, FK consistency, and depth distillation:

$$\mathcal{L} = \mathcal{L}_{\text{diff}} + \lambda_{\text{fk}} \mathcal{L}_{\text{fk}} + \lambda_{\text{depth}} \mathcal{L}_{\text{depth}} \tag{15}$$

IV. EXPERIMENTS

A. Experimental Setup

a) Implementation Details: PEAfowl uses a single-step observation history and predicts an action chunk with horizon $H = 64$, of which 32 steps are executed in a receding-horizon manner. RGB-D inputs are resized to 320×256 from four camera views. The geometry branch uses feature strides $(8, 16, 32, 64)$, while the diffusion decoder attends to the stride-16 and stride-32 levels. Per-token depth distributions are predicted over $B = 128$ linearly spaced bins in $[0.01, 1.2]$ m, and cross-view aggregation uses top-$K = 16$ neighbors with distance-softmax temperature $\tau = 0.08$ m. The pairwise RGB-D fusion block contains one four-head attention layer followed by an FFN with hidden dimension 128. Training uses AdamW with learning rate 1.41×10^{-4}, weight decay 5×10^{-4}, batch size 16, and 4×10^5 updates. Training uses a 1,000-step DDPM scheduler with squared-cosine beta schedule, while inference uses a DPMSolverMultistep scheduler with 10 denoising steps.

TABLE I

SAME-HARDWARE RUNTIME ANALYSIS. LATENCY IS MEASURED ON A SINGLE NVIDIA RTX 4090 WITH BATCH SIZE ONE, FOUR RGB-D VIEWS AT 320×256, 10 WARM-UP ITERATIONS, 100 TIMED ITERATIONS, AND 10 DIFFUSION DENOISING STEPS.

Item	SEM	PEAfowl
Total params (M)	183.9	740.0
Trainable params (M)	183.9	312.4
Peak GPU memory (GB)	0.95	3.04
Preprocess latency (ms)	23.2	23.9
Model latency (ms)	291.3	301.2
Total latency (ms)	314.5	325.1
Throughput (Hz)	3.18	3.08

As shown in Tab. I, PEAfowl increases total latency by 10.6 ms, corresponding to a 3.4% overhead, while retaining a comparable policy-query frequency. The increased memory mainly comes from frozen CLIP encoders used by the language-guided readout.

b) Simulation Setup: We evaluate our method in simulation on RoboTwin 2.0 [3], a scalable benchmark for bimanual manipulation. Two environment settings are used: Clean and Domain-Randomized (DR). The clean setting uses fixed visual and physical parameters with templated instructions for in-distribution evaluation, while the DR setting applies strong structured perturbations, e.g., distractors, lighting, texture, tabletop height, and instruction paraphrases, to stress-test robustness and cross-scene generalization. All simulation experiments use the Aloha-AgileX embodiment with a 4-camera RGB-D setup. Unless otherwise specified, results are reported on 9 tasks spanning short-, mid-, and long-horizon interactions, with 50 demonstrations per task. VLA models are trained jointly across all tasks, while visuomotor policies are trained separately for each task.

c) Real-World Setup: Experiments are conducted on a dual-arm AgileX Piper platform that mirrors the simulation configuration. Our real-robot setting includes 6 tasks: 4 adapted from simulation for sim-to-real evaluation, and 2 real-only tasks included in the real-world training data. We collect 100 high-quality demonstrations per task via VR teleoperation. Methods are initialized from simulation-pretrained models and then fine-tuned separately for each task on the corresponding real-world dataset. The real-world protocol follows representative real-robot manipulation evaluations such as OpenVLA [2] and DP3 [21], where each task is evaluated with 10 real-robot trials.

We use four Intel RealSense D435 RGB-D cameras with aligned depth-to-color streams, each providing 424×240 RGB and depth at 30 FPS, and camera extrinsics are provided by external rig calibration. During preprocessing, we apply a calibration-preserving remap to a fixed target resolution of 320×256 with a fixed destination intrinsic matrix. Compared with naive resizing, this remap effectively crops the usable

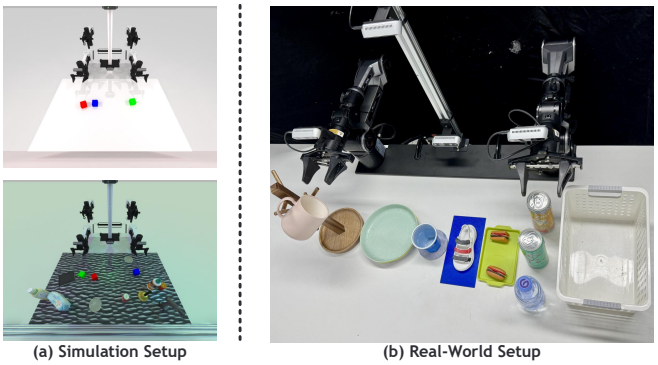


Fig. 4. **Simulation and real-world setups.** (a) RoboTwin 2.0 simulation (Aloha-AgileX, 4-camera RGB-D) under Clean (top) and Domain-Randomized (bottom) settings. (b) Dual-arm AgileX Piper with a 4-camera rig.

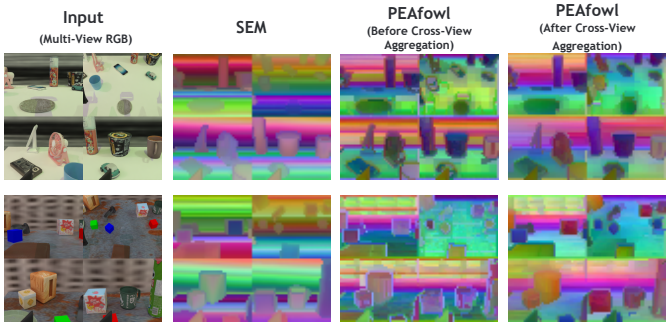


Fig. 5. **Cross-view token consistency.** Compared with SEM and PEAfowl pre-aggregation, PEAfowl post-aggregation yields more coherent cross-view clusters for the same physical regions across cameras.

source region, leading to a narrower field of view and reduced cross-view overlap. As a result, real-robot multi-view correspondence and fusion become more challenging. Fig. 4 provides an overview of the real-world setup.

d) Baselines: We compare PEAfowl with visuomotor and bimanual VLA baselines. Visuomotor policies include ACT [22], DP [23], and DP3 [21], trained separately per task. VLA baselines trained jointly across tasks include π_0 [5], RDT [6], and SEM [4]. All baselines use the same observation setup, training splits, and sim-to-real fine-tuning protocol as PEAfowl.

e) Metrics: In simulation, we run 100 trials per task per setting, and on real-robot experiments 10 trials per task. We report **Success Rate**, the fraction of trials that finish the task within the episode time limit.

B. Results and Analysis

a) Simulation Results: As shown in Tab. II, under the RoboTwin 2.0 DR setting, PEAfowl achieves an average success rate of **47.1%** across 9 tasks, consistently outperforming all baselines. In contrast, task-specific visuomotor policies trained per task rarely succeed on these challenging tasks, suggesting that 50 demonstrations are insufficient for these task-specific baselines to handle appearance and layout variations.

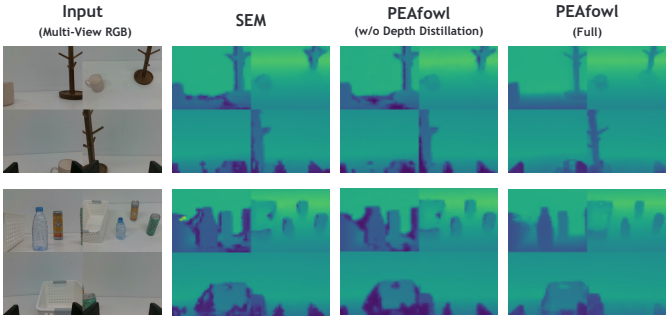


Fig. 6. **Depth-distribution predictions (real robot).** Predicted per-token depth distributions for SEM, PEAfowl w/o depth distillation, and full PEAfowl. PEAfowl produces cleaner predictions than SEM, and training-only depth distillation further sharpens and completes depth under noisy commodity sensors.

TABLE II
SIMULATION RESULTS ON ROBOTWIN 2.0. SUCCESS RATE (%) UNDER CLEAN AND DR (DOMAIN-RANDOMIZED) SETTINGS. TASK ABBREVIATIONS: OM=**OPEN MICROWAVE**, SB3=**STACK BLOCKS THREE**, SW3=**STACK BOWLS THREE**, BR-RGB=**BLOCKS RANKING RGB**, BR-SIZE=**BLOCKS RANKING SIZE**, HM=**HANGING MUG**, OL=**OPEN LAPTOP**, PBF=**PLACE BURGER FRIES**, PEC=**PLACE EMPTY CUP**.

Horizon	Task	DP	DP3	ACT	π_0	RDT	SEM	PEAfowl
				Clean				
Long	OM	5	61	**86**	25	9	17	70
	SB3	0	1	0	2	0	29	**72**
	SW3	63	57	48	47	19	68	**79**
	BR-RGB	0	3	1	16	0	55	**84**
	BR-Size	1	2	0	4	0	21	**27**
	Long Avg.	13.8	24.8	27.0	18.8	5.6	38.0	**66.4**
Mid	HM	8	17	7	6	0	24	**41**
	OL	49	**82**	56	38	25	75	77
	Mid Avg.	28.5	49.5	31.5	22.0	12.5	49.5	**59.0**
Short	PBF	72	72	49	28	19	93	**95**
	PEC	37	65	61	32	23	77	**81**
	Short Avg.	54.5	68.5	55.0	30.0	21.0	85.0	**88.0**
	Avg.	26.1	40.0	34.2	22.0	10.6	51.0	**69.6**
				DR				
Long	OM	0	22	17	8	8	25	**34**
	SB3	0	0	0	15	0	1	**34**
	SW3	30	8	8	48	6	41	**66**
	BR-RGB	0	0	0	4	0	15	**47**
	BR-Size	0	1	0	10	0	0	**14**
	Long Avg.	6.0	6.2	5.0	17.0	2.8	16.4	**39.0**
Mid	HM	1	9	2	6	3	18	**26**
	OL	32	61	18	45	31	61	**68**
	Mid Avg.	16.5	35.0	10.0	25.5	17.0	39.5	**47.0**
Short	PBF	38	21	22	31	9	46	**65**
	PEC	1	5	3	32	3	10	**70**
	Short Avg.	19.5	13.0	12.5	31.5	6.0	28.0	**67.5**
	Avg.	11.3	14.1	7.8	22.1	6.7	24.1	**47.1**

Among multi-task bimanual VLA baselines, π_0 achieves an average of 22.1% under the DR setting, indicating that large-scale vision-language and robot-data pretraining improve cross-scene robustness, whereas RDT underperforms under heavy randomization. SEM is the strongest baseline, which leverages 3D spatial position embeddings along with a joint-centric state encoder and action decoder.

PEAfowl improves over SEM by **23.0 pp**, with particularly

TABLE III
IN-FAMILY HELD-OUT TRANSFER RESULTS. SUCCESS RATES (%) ON TWO HELD-OUT ROBOTWIN TASK VARIANTS UNDER THE CLEAN AND DR SETTINGS. MODELS ARE TRAINED ON THE CORRESPONDING NINE-TASK SUITE AND EVALUATED ON SB2 AND SW2. TASK ABBR.: SB2=**STACK BLOCKS TWO**, SW2=**STACK BOWLS TWO**.

Task	Setting	π_0	RDT	SEM	PEAfowl
SB2	Clean	25	0	49	**74**
	DR	34	0	14	**51**
SW2	Clean	83	38	93	**97**
	DR	79	23	65	**82**

TABLE IV
REAL-WORLD RESULTS. SUCCESS RATES (%) ON THE DUAL-ARM PLATFORM. TASK ABBR.: PS=**PLACE SHOE**, PBD=**PUT BOTTLES DUSTBIN**.

Task	PEAfowl	PEAfowl w/o DD	SEM
SW3	**100 (10/10)**	70 (7/10)	30 (3/10)
HM	**30 (3/10)**	10 (1/10)	0 (0/10)
PBF	**60 (6/10)**	30 (3/10)	10 (1/10)
PEC	**80 (8/10)**	50 (5/10)	20 (2/10)
PS	**60 (6/10)**	30 (3/10)	10 (1/10)
PBD	**80 (8/10)**	30 (3/10)	0 (0/10)
Avg.	**68.3**	36.7	11.7

large gains on long-horizon and occlusion-heavy tasks. Notably, on visually similar yet instruction-distinct tasks (**Stack Blocks Three** vs. **Blocks Ranking RGB**), π_0 and SEM exhibit task-specific biases, whereas PEAfowl performs well on both, suggesting more effective instruction-conditioned visual readout and robustness to clutter and occlusions.

We further visualize multi-view token embeddings before and after cross-view aggregation using t-SNE as shown in Fig. 5. After aggregation, tokens associated with visually corresponding regions across views form more coherent clusters, qualitatively supporting that geometry-guided fusion promotes cross-view feature alignment under distractors and viewpoint changes.

In the clean setting, which evaluates in-distribution task performance, PEAfowl achieves **69.6%** average success, surpassing SEM by **18.6 pp**. Improvements are most obvious on long-horizon tasks requiring consistent spatial awareness under repeated occlusions and viewpoint changes, while gains on short-horizon tasks are smaller due to high baseline performance. Consistent with the DR evaluation, PEAfowl better identifies instruction-relevant targets and relations in referentially ambiguous scenes.

Finally, we evaluate in-family held-out transfer on two additional tasks not included in the nine-task training set in Tab. III. PEAfowl consistently outperforms all baselines under both clean and DR settings. Notably, for **Stack Blocks Two** under DR, SEM drops from 49% to 14%, while PEAfowl maintains 51%, suggesting better transfer to unseen but related bimanual tasks.

b) Real-World Results: Tab. IV compares PEAfowl with SEM, the strongest baseline in simulation, and further ablates training-only depth distillation (DD). PEAfowl outperforms SEM on all 6 real-robot tasks, indicating improved real-robot performance under commodity RGB-D sensing. DD brings additional improvements, particularly on depth-sensitive tasks such as **Put Bottles Dustbin** and **Hanging Mug**. Fig. 6 visualizes an example of depth-distribution prediction, suggesting that distillation injects geometric priors from a pretrained depth teacher into the depth-distribution head.

We also observe that SEM suffers depth-loss divergence during real-robot fine-tuning (cf. Section IV-C), highlighting the difficulty of learning stable depth-aware representations from noisy sensor depth alone. In contrast, PEAfowl's local RGB-D pairwise fusion and cross-view 3D neighbor aggregation mitigate unreliable depth and stabilize multi-view spatial representation learning. Qualitative rollouts are provided in the supplementary video.

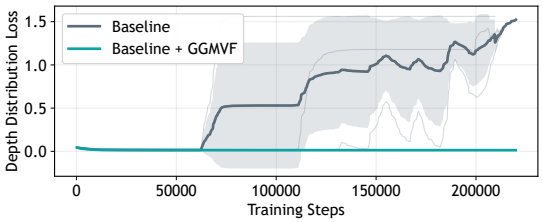


Fig. 7. Depth-loss stability. Under multi-task training, the Baseline's depth loss diverges, while adding GGMVF stabilizes depth-distribution learning.

TABLE V
ABLATIONS ON ROBOTWIN 2.0. SUCCESS RATE (%) ON NINE TASKS UNDER CLEAN AND DR. TASK ABBREVIATIONS FOLLOW TABLE II.

Task	PEAfowl	w/1-step SimAttn	w/o VG-Text	Baseline + GGMVF	Baseline
Clean					
OM	**70**	44	65	45	17
SB3	**72**	70	39	21	1
SW3	**79**	71	75	77	50
BR-RGB	**84**	75	62	69	44
BR-Size	**27**	26	**27**	21	16
HM	**41**	38	40	39	29
OL	**77**	74	76	76	76
PBF	95	**98**	**98**	94	95
PEC	**81**	79	79	77	74
Avg.	**69.6**	63.9	62.3	57.7	44.7
DR					
OM	**34**	31	32	26	17
SB3	**34**	18	26	7	1
SW3	**66**	61	61	61	50
BR-RGB	**47**	35	31	34	11
BR-Size	**14**	1	3	7	0
HM	**26**	19	20	23	14
OL	**68**	65	**68**	**68**	**68**
PBF	**65**	49	49	44	**65**
PEC	**70**	46	60	52	56
Avg.	**47.1**	36.1	38.9	35.8	31.3

C. Ablation Studies

We ablate PEAfowl on RoboTwin 2.0 by removing individual components from the full model and by adding our geometry-guided perception to a simple visuomotor baseline. The baseline uses the same RGB-D encoders and SEM-style diffusion decoder as PEAfowl, but excludes both the language-guided readout and the geometry-guided operations, including

TABLE VI
AVERAGE MULTI-VIEW ABLATION UNDER THE DR SETTING. SUCCESS RATE (%). "W/O FRONT" REMOVES THE FRONT VIEW; "HEAD-ONLY" REMOVES THE FRONT AND BOTH WRIST VIEWS.

Setting	PEAfowl	w/o Front	Head-only
Avg.	47.1	35.0	26.8

pairwise RGB-D fusion and cross-view 3D neighbor aggregation. We evaluate three variants: (i) replacing the Perceiver-style iterative readout with OTTER's one-step similarity readout (**w/1-step SimAttn**); (ii) removing vision-grounded text tokens and conditioning the policy only on instruction-summary tokens (**w/o VG-Text**); and (iii) adding the full geometry-guided module to the baseline (**Baseline+GGMVF**).

As shown in Tab. V, **w/1-step SimAttn** yields a small degradation under the clean setting but a substantially larger drop under DR, suggesting the effectiveness of iterative, text-aware evidence accumulation under clutter and scene shifts. **w/o VG-Text** also lowers success, with more obvious drops on attribute- and reference-sensitive tasks, supporting the contribution of text-aware visual extraction. Finally, the baseline suffers from depth-loss divergence and collapsed depth distributions. This instability is consistently observed across independent training runs with different random seeds under the same settings (Fig. 7). **Baseline+GGMVF** remains stable and improves long-horizon performance, indicating that our geometry-guided pathway contributes to more stable depth-aware multi-task learning.

We further ablate camera views under the DR setting. As shown in Tab. VI, removing the front view reduces the average success rate from 47.1% to 35.0%, and using only the head view further reduces it to 26.8%, confirming the importance of complementary front and wrist observations for bimanual manipulation.

V. CONCLUSIONS

In this paper, we proposed PEAfowl, a perception-enhanced multi-view VLA policy for bimanual manipulation that improves robustness under clutter, occlusions, and scene variations in the evaluated settings. PEAfowl couples geometry-guided fusion for depth-aware cross-view feature aggregation with a Perceiver-style text-as-query readout over frozen CLIP features for improved instruction-conditioned visual extraction. Across RoboTwin 2.0 and real-robot evaluations, PEAfowl improves over the evaluated bimanual VLA and visuomotor baselines, with ablations highlighting the contributions of geometry-guided fusion, language-guided readout, depth distillation, and multi-view sensing.

REFERENCES

[1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th CoRL*, vol. 229. PMLR, 2023, pp. 2165–2183.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th CoRL*, vol. 270. PMLR, 2025, pp. 2679–2713.

[3] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, W. Deng, Y. Guo, T. Nian, X. Xie, Q. Chen, K. Su, T. Xu, G. Liu, M. Hu, H. ang Gao, K. Wang, Z. Liang, Y. Qin, X. Yang, P. Luo, and Y. Mu, "RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," in *Proceedings of the 43rd ICML*, 2026.

[4] X. Lin, T. Lin, L. Huang, H. Xie, Y. Jin, K. Li, and Z. Su, "SEM: Enhancing spatial understanding for robust robot manipulation," arXiv preprint arXiv:2505.16196, 2025.

[5] K. Black, N. Brown, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter *et al.*, "π_0: A Vision-Language-Action Flow Model for General Robot Control," in *Proceedings of RSS*, LosAngeles, CA, USA, June 2025.

[6] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1B: A diffusion foundation model for bimanual manipulation," in *ICLR*, 2025.

[7] H. Huang, F. Liu, L. Fu, T. Wu, M. Mukadam, J. Malik, K. Goldberg, and P. Abbeel, "OTTER: A vision-language-action model with text-aware visual feature extraction," in *Proceedings of the 42nd ICML*, vol. 267. PMLR, 2025.

[8] A. Jaegle, S. Borgeaud, J.-B. Alayrac, C. Doersch, C. Ionescu, D. Ding, S. Koppula, D. Zoran, A. Brock, E. Shelhamer, O. J. Henaff, M. Botvinick, A. Zisserman, O. Vinyals, and J. Carreira, "Perceiver IO: A general architecture for structured inputs & outputs," in *ICLR*, 2022.

[9] M. Liu, Z. Zhu, X. Han, P. Hu, H. Lin, X. Li, J. Chen, J. Xu, Y. Yang, Y. Lin, X. Li, Y. Yu, W. Zhang, T. Kong, and B. Kang, "Manipulation as in simulation: Enabling accurate geometry perception in robots," in *ICLR*, vol. 2026, 2026, pp. 11 238–11 271.

[10] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny, "Vggt: Visual geometry grounded transformer," in *Proceedings of CVPR*, June 2025, pp. 5294–5306.

[11] Y. Seo, J. Kim, S. James, K. Lee, J. Shin, and P. Abbeel, "Multi-view masked world models for visual robotic manipulation," in *Proceedings of the 40th ICML*, vol. 202. PMLR, 2023, pp. 30 613–30 632.

[12] S. Qian, K. Mo, V. Blukis, D. F. Fouhey, D. Fox, and A. Goyal, "3d-mvp: 3d multiview pretraining for manipulation," in *Proceedings of CVPR*, 2025, pp. 22 530–22 539.

[13] A. Goyal, V. Blukis, J. Xu, Y. Guo, Y.-W. Chao, and D. Fox, "RVT-2: Learning Precise Manipulation from Few Demonstrations," in *Proceedings of RSS*, Delft, Netherlands, July 2024.

[14] Q. Fan, Y. Cai, C. Li, W. He, X. Zheng, T. Lu, B. Liang, and S. Wang, "Neugrasp: Generalizable neural surface reconstruction with background priors for material-agnostic object grasp detection," in *IEEE ICRA*, 2025, pp. 3197–3203.

[15] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, J. Gu, Z. Wang, Y. Ding, B. Zhao, D. Wang, and X. Li, "SpatialVLA: Exploring Spatial Representations for Visual-Language-Action Models," in *Proceedings of RSS*, LosAngeles, CA, USA, June 2025.

[16] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proceedings of ECCV*, 2024.

[17] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of CVPR*, 2016, pp. 770–778.

[18] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th ICML*, vol. 139. PMLR, 2021.

[19] M. Lan, C. Chen, Y. Ke, X. Wang, L. Feng, and W. Zhang, "ClearCLIP: Decomposing CLIP representations for dense vision-language inference," in *Proceedings of ECCV*, 2024.

[20] T. Bachlechner, B. P. Majumder, H. Mao, G. Cottrell, and J. McAuley, "ReZero is all you need: Fast convergence at large depth," in *Proceedings of the 37th UAI*, vol. 161. PMLR, 2021.

[21] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu, "3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations," in *Proceedings of RSS*, Delft, Netherlands, July 2024.

[22] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," in *Proceedings of RSS*, Daegu, Republic of Korea, July 2023.

[23] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," in *Proceedings of RSS*, Daegu, Republic of Korea, July 2023.

TABLE VII
IMPLEMENTATION DETAILS OF PEAFOWL.

Category	Item	Setting
Observation	History length	$H_{\text{obs}} = 1$
Observation	RGB-D input resolution	320×256
Observation	Number of camera views	4
Policy output	Prediction horizon	$H = 64$
Policy output	Valid action chunk	32 steps
Token dimension	Hidden dimension	$d = 256$
RGB encoder	Backbone	Swin-T initialized from Grounding-DINO
Depth encoder	Backbone	ResNet-34
Geometry branch	Feature strides	$(8, 16, 32, 64)$
Geometry branch	Decoder-attended levels	stride-16 and stride-32
Depth distribution	Depth bins	$B = 128$
Depth distribution	Depth range	$[0.01, 1.2]$ m
Depth distribution	Bin interval	≈ 9.4 mm
Depth supervision	Token-validity threshold	0.5
Cross-view aggregation	Neighbor cap	$K = 16$
Cross-view aggregation	Distance temperature	$\tau = 0.08$ m
Cross-view aggregation	Residual gate initialization	0.5
RGB-D pair fusion	Attention block	1 block, 4 heads
RGB-D pair fusion	Fusion dimension	$d_{\text{dep}} = 32$
RGB-D pair fusion	FFN hidden dimension	128
Language branch	CLIP backbone	frozen CLIP ViT-L/14
Language branch	Retained text tokens	$K_{\text{txt}} = 64$
Language branch	Perceiver-style latent blocks	$M = 3$ per view
Language branch	Readout tokens	$R = 64$ per view
Language branch	ReZero gate initialization	0
Action decoder	Diffusion decoder blocks	6
Diffusion training	Scheduler	DDPM
Diffusion training	Timesteps	1000
Diffusion training	Beta schedule	squared cosine
Diffusion training	Prediction type	sample prediction
Diffusion inference	Scheduler	DPMSolverMultistep
Diffusion inference	Denoising steps	10
Main simulation optimization	Optimizer	AdamW
Main simulation optimization	Batch size	16
Main simulation optimization	Training updates	4×10^5
Main simulation optimization	Base learning rate	1.41×10^{-4}
Main simulation optimization	Weight decay	5×10^{-4}
Main simulation optimization	RGB-backbone LR multiplier	$0.1\times$
Main simulation optimization	Warmup	500 iterations
Main simulation optimization	LR decay	$\times 0.1$ at 90% training

APPENDIX A
ADDITIONAL IMPLEMENTATION DETAILS

A. Core Model and Training Settings

Table VII summarizes the core implementation settings. PEAfowl uses a single-step observation history and predicts an action chunk with horizon $H = 64$. During deployment, the policy is queried in a receding-horizon manner. The geometry-guided branch predicts per-token depth distributions over $B = 128$ linearly spaced depth bins in $[0.01, 1.2]$ m, corresponding to a bin interval of approximately 9.4 mm. Cross-view 3D aggregation uses up to $K = 16$ nearest tokens from other camera views (or all available candidates when fewer than 16 exist) and a distance-softmax temperature $\tau = 0.08$ m.

B. Pairwise RGB-D Token Fusion

The pairwise RGB-D token fusion module implements the local fusion in Eq. 3. For each view v and token index n, the co-located RGB and depth tokens are projected to the common fusion dimension $d_{\text{dep}} = 32$ and processed as a length-2 sequence. The module consists of one four-head self-attention layer followed by a feed-forward network with hidden dimension 128. We apply this local fusion only in the depth-distribution branch; the main RGB stream remains unchanged before cross-view aggregation.

C. Depth Distillation Targets

For real-robot fine-tuning, the pretrained Camera Depth Model (CDM) is used only as an offline teacher. Given the raw RGB-D observation $(\mathbf{I}^{(v)}, \mathbf{D}^{(v)})$, CDM produces a refined depth map $\tilde{\mathbf{D}}^{(v)}$. The policy input remains the raw depth map $\mathbf{D}^{(v)}$ during both training and inference; the refined CDM output is used only to construct supervision for the predicted per-token depth distribution.

For each feature level, the refined pixel depths are first converted to soft two-hot depth-bin targets and then average-pooled according to the corresponding token stride, as described in Sec. III-E0b. The valid-pixel fraction in each pooled region defines a token weight $\omega_n^{(v)} \in [0, 1]$: fractions below the validity threshold of 0.5 are set to zero, while retained tokens keep their continuous valid-pixel fraction. The predicted distributions are supervised by the validity-weighted soft-label BCE loss in Eq. 14.

The depth-distillation coefficient is $\lambda_{\text{depth}} = 1.0$. The diffusion and forward-kinematics regression terms use the same configured state-wise regression weights; the complete objective is given in Eq. 15.

D. Qualitative CDM Depth Refinement

Fig. 8 qualitatively compares raw Intel RealSense D435 depth maps with the corresponding CDM-refined maps. The examples illustrate how the offline teacher modifies missing or noisy regions used to construct token-level depth targets. CDM is used only to generate training supervision for the depth-distribution head and is not executed during policy inference.

APPENDIX B
ADDITIONAL DEPLOYMENT DETAILS

The profiling setup in Tab. I uses a single NVIDIA RTX 4090 GPU, batch size one, four 320×256 RGB-D views, 10 warm-up iterations, 100 timed iterations, and 10 diffusion denoising steps. Under this setup, SEM and PEAfowl have mean/p90 total latencies of 314.5/319.0 ms and 325.1/332.0 ms, respectively. PEAfowl therefore adds 10.6 ms of mean latency (3.4%) and increases peak allocated memory from 0.95 GB to 3.04 GB. CDM is excluded because it is used only for training-target generation. During real-robot deployment, the low-level controller runs at 25 Hz and the high-level policy is queried at 1 Hz, so the measured policy call remains within the 1 s query interval.

TABLE VIII

ROBUSTNESS RESULTS UNDER THE SEPARATELY TRAINED ROBUSTNESS-STUDY CONFIGURATION. SUCCESS RATE (%) ON ROBOTWIN 2.0. SEM IS REPRODUCED UNDER THE SAME TRAINING SETTING AS A MATCHED NO-PERTURBATION REFERENCE. PEAFOWL IS FURTHER EVALUATED UNDER CALIBRATION AND DEPTH PERTURBATIONS WITHOUT RETRAINING. EACH RESULT IS AVERAGED OVER NINE TASKS WITH 100 ROLLOUTS PER TASK. PARENTHESES INDICATE THE CHANGE FROM THE UNPERTURBED PEAFOWL CHECKPOINT UNDER THE SAME SPLIT.

Method	Perturbation	Level	Clean SR	DR SR
SEM	None	–	46.3	11.8
PEAfowl	None	–	**72.4**	**38.3**
PEAfowl	Calibration	Mild	66.4 (−6.0)	36.2 (−2.1)
PEAfowl	Calibration	Medium	49.2 (−23.2)	31.9 (−6.4)
PEAfowl	Calibration	Severe	32.4 (−40.0)	25.0 (−13.3)
PEAfowl	Gaussian depth noise	5 mm	74.7 (+2.3)	38.2 (−0.1)
PEAfowl	Gaussian depth noise	10 mm	71.2 (−1.2)	38.0 (−0.3)
PEAfowl	Gaussian depth noise	20 mm	72.4 (0.0)	37.8 (−0.5)
PEAfowl	Missing depth	10%	43.2 (−29.2)	24.8 (−13.5)
PEAfowl	Missing depth	30%	10.8 (−61.6)	20.9 (−17.4)
PEAfowl	Missing depth	50%	7.2 (−65.2)	7.1 (−31.2)



(a) Put Bottles Dustbin from our real-robot data.



(b) Hanging Mug from our real-robot data.



(c) Separately collected cluttered multi-object scene.

Fig. 8. **Qualitative comparison between raw Intel RealSense D435 depth and CDM-refined depth.** In each panel, the top row shows RGB and raw depth; the bottom row shows CDM-refined depth and the pixel-wise difference map. (a) and (b) are from real-robot data. (c) is a separately collected cluttered scene for visualization only and is not used in experimental results.

APPENDIX C
ADDITIONAL ROBUSTNESS ANALYSIS

A. Training Configuration for Robustness Experiments

The PEAfowl robustness experiments use separately trained Clean and DR checkpoints rather than the main-result checkpoints. For each split, the corresponding PEAfowl robustness checkpoint is trained on two 48 GB GPUs with a per-GPU batch size of 16, a global batch size of 32, and a learning rate of 2×10^{-4}. The Clean configuration uses a maximum of 152,712 updates over 28 epochs, whereas the DR configuration uses a maximum of 148,980 updates over 26 epochs. Since this configuration differs from the main-result training setting, we use the results in this section only for within-checkpoint sensitivity analysis: each perturbed result is compared with the no-perturbation result from the corresponding split-specific PEAfowl checkpoint.

Each PEAfowl robustness checkpoint is trained on the same nine RoboTwin tasks with 50 demonstrations per task. The policy uses the same four RGB-D views, proprioceptive state, and 64-step action chunk as in the main experiments. Each evaluation setting is tested in closed-loop rollouts over the nine tasks, with 100 rollouts per task.

As matched no-perturbation references, we also reproduce SEM under the same task set, demonstration budget, observation setup, two-GPU training setup, batch size, and learning rate. The reproduced SEM results are 46.3% on Clean and 11.8% on DR, respectively. Thus, the key comparisons in this section are PEAfowl versus SEM under the same no-perturbation robustness-study setting, and perturbed PEAfowl versus the no-perturbation PEAfowl result from the corresponding split-specific checkpoint.

B. Perturbation Protocol

After training, we evaluate each policy under controlled perturbations to camera calibration and depth observations. During evaluation, the environment dynamics, RGB observations, task initialization distribution, and policy weights remain fixed for a given checkpoint. Only the camera parameters or depth maps used by the policy are perturbed.

For calibration perturbation, each camera independently receives one fixed perturbation at the beginning of an episode, and the perturbation remains constant throughout the episode. The translation, rotation, focal-length, and principal-point perturbation variables are independently sampled from zero-mean Gaussian distributions. This models persistent calibration bias rather than frame-wise camera jitter:

$$\mathbf{T}'_{\text{world}\to\text{cam}} = \Delta\mathbf{T}\,\mathbf{T}_{\text{world}\to\text{cam}}, \tag{16}$$

$$f'_x = f_x(1+\epsilon_{f_x}), \quad f'_y = f_y(1+\epsilon_{f_y}). \tag{17}$$

$$c'_x = c_x + \epsilon_{c_x}, \quad c'_y = c_y + \epsilon_{c_y}. \tag{18}$$

The mild, medium, and severe settings respectively use translation standard deviations of $(5, 15, 30)$ mm, rotation standard deviations of $(1, 3, 5)^\circ$, focal-length standard deviations of $(0.5, 1.5, 3.0)\%$, and principal-point standard deviations of $(2, 5, 10)$ pixels.

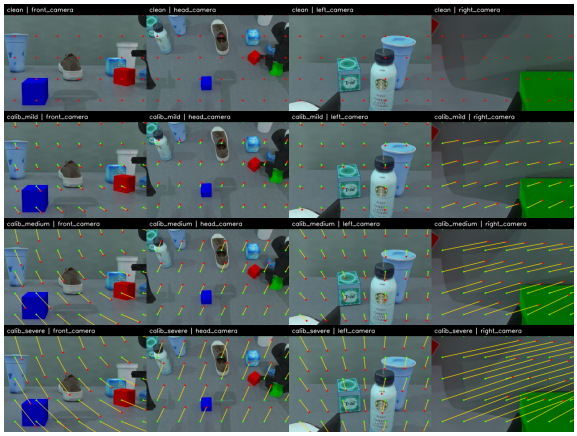


Fig. 9. **Representative visualization of calibration perturbations.** Green and red markers denote original and perturbed projections, respectively, and yellow line segments show the induced image-space projection displacement. Each camera receives one episode-level perturbation that remains fixed throughout the episode, modeling persistent calibration bias rather than frame-wise jitter.

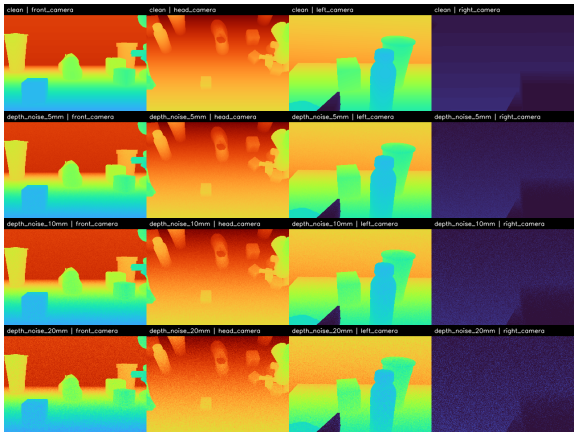


Fig. 10. **Representative depth observations under additive Gaussian noise.** Rows from top to bottom correspond to depth-noise standard deviations of 0, 5, 10, and 20 mm. Noise is added only to valid depth pixels, while originally invalid pixels remain invalid.

For additive depth noise, zero-mean Gaussian noise is added only to valid depth pixels:

$$d' = \max(d + \epsilon_d, 0), \qquad \epsilon_d \sim \mathcal{N}(0, \sigma_d^2), \tag{19}$$

where $\sigma_d \in \{5, 10, 20\}$ mm. Existing invalid pixels remain invalid.

For missing-depth corruption, we randomly set 10%, 30%, or 50% of originally valid depth pixels to zero while keeping the RGB observations and camera parameters unchanged. The evaluation script uses a fixed default perturbation seed of 2026 for these controlled perturbations.

C. Robustness Results

Table VIII reports results for the separately trained robustness-study checkpoints. Under matched no-perturbation settings, PEAfowl is 26.1 pp higher than the reproduced SEM reference on Clean and 26.5 pp higher on DR. The remaining rows evaluate each PEAfowl checkpoint under calibration

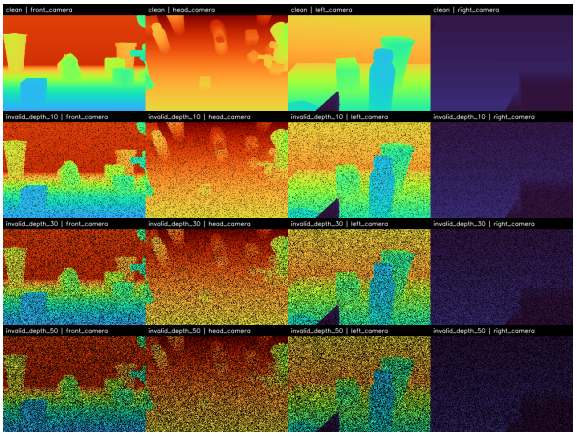


Fig. 11. **Representative depth observations under missing-depth corruption.** Rows from top to bottom correspond to missing-depth ratios of 0%, 10%, 30%, and 50%. Missing-depth corruption randomly sets originally valid depth pixels to zero while leaving RGB observations and camera parameters unchanged.

and depth perturbations without retraining. Because these checkpoints use a different batch size and learning rate from the main-result checkpoints, their absolute success rates are interpreted only within this robustness-study configuration.

Within the evaluated ranges, additive depth noise produces smaller changes than severe missing-depth corruption or calibration error. This sensitivity is consistent with PEAfowl's use of expected 3D token anchors for cross-view retrieval, but the diagnostic experiment does not isolate a causal mechanism.

APPENDIX D
REAL-WORLD EVALUATION PROTOCOL

A. Autonomous Trial Protocol

All formal real-world trials are fully autonomous. Once a trial starts, no human intervention is allowed until the trial terminates as either success or failure. Human intervention is used only between trials to reset the scene.

A trial is counted as a failure if any of the following occurs:

- the robot requires manual correction during execution;
- the scene requires manual recovery before the trial terminates;
- the robot fails to complete the task within the 2-min execution horizon.

PEAfowl does not include an explicit failure detector or recovery planner. Since the policy is executed from current multi-view RGB-D observations, it may recover from some intermediate execution errors. For example, if a missed grasp leaves the object in a state that remains reachable and observable, the policy may reattempt the manipulation from the new observation. However, if the task is not completed within the allowed 2-min horizon, the trial is still counted as a failure.